TA-CBF: Complete Technical Deep-Dive

From a Single Neuron to the Full QP Controller — Everything Explained

Architecture · Training · Inference · Docs Map · ICRA Assessment · Q&A

Table of Contents

> **Important corrections from the previous PDF:** (1) f_θ is a PLAIN MLP, NOT a ResNet. (2) STATE_DIM = 3 (x, y, z) — task is 3D, effectively 2D in XY at fixed Z=0.44 m. (3) The full Lyapunov formula is V = ‖e‖²·(1+δ·corr), not just ‖e‖². (4) CBF input is 67D (3+64), not 66D.

PART I — ARCHITECTURE IN FULL DETAIL

I.1 State Space: What x Actually Is

The **state** $x \in \mathbb{R}^3$ is the 3D needle-tip position measured in meters in the robot's base frame:

```
x = [x₁, x₂, x₃] = [x_coordinate, y_coordinate, z_coordinate]

Typical range from demos:
  x₁ ∈ [0.44, 0.55] m    (forward, left-right)
  x₂ ∈ [0.06, 0.12] m    (lateral)
  x₃ = 0.44 m  (fixed! — needle stays at this height)

Because x₃ never changes meaningfully, the task is effectively 2D in (x₁, x₂).
All obstacle shapes are defined in the XY plane; point clouds are 2D (K, 2) arrays.
```

Before feeding x into any network, it is **normalized** using the mean and std computed from training data:

```
x̃ = (x − μ) / σ    where μ ∈ ℝ³, σ ∈ ℝ³

Example: μ = [0.49, 0.08, 0.44],  σ = [0.028, 0.018, 0.003]

x = [0.50, 0.10, 0.44] → x̃ = [(0.50−0.49)/0.028, (0.10−0.08)/0.018, (0.44−0.44)/0.003
                            = [0.357, 1.111, 0.0]

This puts all features on a similar scale so no single dimension dominates learning.
```

I.2 Task Progress s ∈ [0, 1]: The "Where Along the Path" Signal

The demo path is stored as R=200 waypoints (from X_START to X_GOAL). Progress s tells each network how far along the task we are:

```
get_progress(x):
  diff  = x[None,:] − ref_path        # shape (1,3) − (200,3) → (200,3)  [broadcast]
  dists = ‖diff‖ over dim 2           # distances to each waypoint → (200,)
  idx   = argmin(dists)               # index of nearest waypoint, 0..199
  s     = idx / 199                   # normalize → [0, 1]

x_ref(s):
  lo = floor(s × 199),  hi = lo+1
  w = s×199 − lo              # fractional weight
  return (1−w)·ref_path[lo] + w·ref_path[hi]   # linear interpolation
```

So s=0 → at start, s=0.5 → halfway, s=1 → at goal. s is recomputed every control step.

I.3 Component A: ProgressConditionedDS — Demo Flow Network f_θ

f_θ(x, s) outputs the desired velocity (m/s) at position x with progress s. It is a **plain MLP** (NOT ResNet — the previous PDF was wrong) with 3 hidden layers of 128 neurons each.

```
Input vector:  inp = [x̃₁, x̃₂, x̃₃, s]  ∈  ℝ⁴   (normalized position + progress)

Network (plain MLP with Tanh):
  Layer 1: Linear(4 → 128) → Tanh      output: h₁ ∈ ℝ¹²⁸
  Layer 2: Linear(128 → 128) → Tanh    output: h₂ ∈ ℝ¹²⁸
  Layer 3: Linear(128 → 128) → Tanh    output: h₃ ∈ ℝ¹²⁸
  Layer 4: Linear(128 → 3)             output: v_ff ∈ ℝ³   [NO activation — velocity

Total parameters: (4×128+128) + (128×128+128)×2 + (128×3+3)
                = 640 + 16,512×2 + 387 = 34,051 weights
```

What does each layer "do" intuitively?

Layer	Input	Output	What it computes
Linear(4→128)+Tanh	$[\tilde{x},s] \in \mathbb{R}^4$	$h_1 \in \mathbb{R}^{128}$	128 non-linear features of position+progress: "am I near a bend?", "am I early/late in task?", etc.
Linear(128→128)+Tanh	$h_1 \in \mathbb{R}^{128}$	$h_2 \in \mathbb{R}^{128}$	Combines features: "am I in the segment that curves upward at progress 0.3?"
Linear(128→128)+Tanh	$h_2 \in \mathbb{R}^{128}$	$h_3 \in \mathbb{R}^{128}$	Higher-level trajectory features: turning direction, curvature
Linear(128→3)	$h_3 \in \mathbb{R}^{128}$	$v_{ff} \in \mathbb{R}^3$	Final velocity vector in 3D. No Tanh so velocity is unbounded.

The Feedback Correction Term

The pure feedforward v_ff may drift. A **spring correction** pulls the needle back to the nearest demo point:

```
v_fb = K_f · (x_ref(s) − x)    where K_f = DEMO_K = 5.0

This is literally a spring with stiffness 5: if x is 10 mm off the demo path in x-dir
v_fb adds 5.0 × 0.010 = 0.05 m/s of correction toward the path.

FINAL DEMO FLOW:
  f_θ(x, s) = v_ff(x̃, s) + v_fb(x, s) = v_ff + 5·(x_ref(s) − x)  ∈ ℝ³
```

Worked Example — Full f_θ computation:

x = [0.500, 0.100, 0.44], s = 0.5 (halfway)

x̃ = [(0.500−0.49)/0.028, (0.100−0.08)/0.018, 0] = [0.357, 1.111, 0]

inp = [0.357, 1.111, 0, 0.5] → through 4 layers → v_ff = [0.015, −0.005, 0.000] m/s

x_ref(0.5) = [0.494, 0.091, 0.440] (interpolated demo midpoint)

v_fb = 5 × ([0.494,0.091,0.44] − [0.500,0.100,0.44]) = 5×[−0.006,−0.009,0] = [−0.030,−0.045,0]

f_θ = [0.015+(−0.030), −0.005+(−0.045), 0] = [**−0.015, −0.050, 0.000**] m/s

→ Net effect: move left (−x) and down in y toward demo path.

I.4 Component B: LyapunovNet — The FULL V_θ Formula

You asked about this specifically. The formula in the code is NOT just V=‖e‖². The full formula is:

```
V(x, x_ref(s), s) = ‖e‖² · (1 + δ · corr(e_norm, s))

where:
  e        = x − x_ref(s)              ← tracking error vector ∈ ℝ³
  ‖e‖²     = e₁² + e₂² + e₃²          ← squared distance from demo path (the quadratic
  δ        = 0.05                      ← small weight on the correction (5%)
  e_norm   = x̃ − x̃_ref(s)            ← NORMALIZED tracking error (for the NN)
```

```
  corr(·)  = Softplus network          ← learned non-negative correction ≥ 0

The correction network:
  Input:  [e_norm₁, e_norm₂, e_norm₃, s]  ∈  ℝ⁴   (normalized error + progress)
  Layer1: Linear(4 → 64) → Softplus       output: ℝ⁶⁴
  Layer2: Linear(64 → 64) → Softplus      output: ℝ⁶⁴
  Layer3: Linear(64 → 1)  → Softplus      output: ℝ¹  (≥ 0 ALWAYS — Softplus is alway
```

Why Softplus (not Tanh) for the correction?

```
Softplus(z) = log(1 + e^z)

Properties:
  Softplus(z) > 0 for ALL z  (always positive, no exceptions)
  Softplus(−∞) → 0,  Softplus(+∞) → ∞
  Softplus(0) ≈ 0.693

Because all layers use Softplus, corr(e_norm, s) ≥ 0 always.
Therefore 1 + δ·corr ≥ 1 + 0 = 1 > 0 always.
Therefore V = ‖e‖² · (something ≥ 1) ≥ ‖e‖² > 0 for e ≠ 0. ✓
```

Total V_θ parameters: (4×64+64)+(64×64+64)+(64×1+1) = 320 + 4,160 + 65 = **4,545**

Why add the correction at all?

> **The ‖e‖² part alone treats all directions equally — a circle.** But the demo path may curve tightly in one direction and barely at all in another. The correction term corr(e, s) allows V to be "larger in some directions than others" at progress s — an ellipsoidal instead of circular level set. With δ=0.05, the correction never dominates (at most 5% of ‖e‖²), so the quadratic base is preserved and all theoretical properties hold. The correction just slightly reshapes the bowl to match the corridor geometry.

Key Property: V = 0 Exactly on the Demo Path

```
When x = x_ref(s):
  e = x − x_ref(s) = [0, 0, 0]
  ‖e‖² = 0
  V = 0 × (1 + δ·corr(...)) = 0  ← EXACTLY ZERO regardless of what corr outputs!

This is why the correction is MULTIPLICATIVE: V(x_ref(s)) = 0 with zero extra computa
```

I.5 How Does V Guarantee Stability Everywhere?

This is a fundamental question. The CLF condition is:

```
V̇(x) = ∇V · ẋ ≤ −α · V(x)

This is a 1st-order ODE in V(t). If this holds, by Grönwall's inequality:
  V(t) ≤ V(0) · e^(−α·t)

So V(t) → 0 exponentially fast as t → ∞.
Since V(x) = 0 iff e = 0 iff x = x_ref(s(t)), this means x converges to the demo path
```

The QP controller finds u that enforces this condition at every step. But does this guarantee global stability everywhere?

> **Honest answer: LOCALLY yes, GLOBALLY only under assumptions.**
>
> The CLF condition is enforced by the QP whenever the QP is feasible. The QP might be **infeasible** if the CBF and CLF constraints conflict so severely that no u can satisfy both. In practice:
>
> - Close to the demo path (small e, small V): both constraints are easy to satisfy simultaneously → $\dot{V} \leq -\alpha V$ holds → **exponential convergence**
> - Near an obstacle (B close to margin): the CBF constraint forces u to point away from obstacle; this might INCREASE V (moving away from demo) → CLF constraint is relaxed by slack ε → **V may temporarily increase**
> - Very far from path AND near obstacle: $\dot{V}$ might be positive, but the system at least stays safe
>
> The **guarantee** is: starting from any safe state, the trajectory remains safe AND converges to a neighborhood of the demo path whenever the obstacle doesn't block the path. If obstacles block the path (pocket trap), the CLF and CBF may both be satisfied simultaneously only by stopping — this is a fundamental limitation of reactive control without a global planner.

The Lyapunov decrease condition as a loss term

```
During training, L_dot enforces: ∇V · f_θ(x, s) ≤ −α · V(e)

This trains f_θ to be a "Lyapunov-compatible" vector field:
the nominal flow alone (without any u) tends to decrease V.
The QP then handles violations when obstacles cause conflicts.
```

I.6 Component C: CompositeBarrier B_φ — Complete Detail

The barrier B_φ(x) measures "how safe is the needle's current position with respect to ALL obstacles simultaneously." It has three sub-parts: encoder, CBF MLP, and smooth-min.

Part A: ObstacleEncoder — Turning a Point Cloud into a Number Vector

Step 0: What is the input point cloud?

```
For each obstacle i, we have a cloud P_i ∈ ℝ^(K×2):
  K = 200 points, each a 2D (x,y) coordinate sampled from obstacle i's interior,
  CENTERED at the obstacle's centroid (so center = (0,0) in the cloud's frame).

Example for the star obstacle:
  P_star = [[ 0.003, −0.008],    ← interior point 1
            [−0.005,  0.012],    ← interior point 2
            [ 0.010,  0.001],    ← interior point 3
            ...200 points...  ]

These 200 points collectively "spell out" the star's shape.
The encoder turns these 200 two-number pairs into a single 64-number vector.
```

How is the cloud sampled? (canonical_interior_cloud)

```
Algorithm:
  1. Pick a random point p in a 1.8×radius bounding box around the shape center
  2. Compute sdf_critical_shape_2d(p, shape) — exact analytic SDF
  3. If sdf < 0 (inside): add (p − center) to the cloud
  4. Repeat until K=96 interior points collected (or max 40,000 attempts)

After sampling, this canonical cloud (in local frame) is STORED ONCE.
During augmentation: apply R·scale transform to the stored cloud → instant augmented
This avoids expensive rejection sampling every training step.
```

Step 1: Per-Point MLP (applied identically to all 200 points)

```
For EACH point pᵢ ∈ ℝ² (i = 1 to 200), the SAME MLP computes:

  h¹ᵢ = ReLU(W₁·pᵢ + b₁)    W₁ ∈ ℝ^(128×2), b₁ ∈ ℝ¹²⁸  → h¹ᵢ ∈ ℝ¹²⁸

  h²ᵢ = ReLU(W₂·h¹ᵢ + b₂)   W₂ ∈ ℝ^(128×128), b₂ ∈ ℝ¹²⁸ → h²ᵢ ∈ ℝ¹²⁸

  h³ᵢ = W₃·h²ᵢ + b₃          W₃ ∈ ℝ^(64×128), b₃ ∈ ℝ⁶⁴  → h³ᵢ ∈ ℝ⁶⁴
                              [NO activation on this last per-point layer]

Shared weights: W₁,b₁,W₂,b₂,W₃,b₃ are the SAME for all 200 points.
Total per-point MLP parameters: (2×128+128)+(128×128+128)+(128×64+64) = 384+16,512+8,
```

> **Concrete single-point example (star obstacle, point $p_1 = [0.003, -0.008]$):**
>
> Layer 1 — one of the 128 neurons (showing neuron #7 only):
> pre-activation = $w_7^T \cdot p + b_7 = [w_{71}, w_{72}] \cdot [0.003, -0.008] + b_7$
> $= 0.42 \times 0.003 + (-1.15) \times (-0.008) + 0.03 = 0.00126 + 0.00920 + 0.03 = 0.040$

h^1_7 = ReLU(0.040) = 0.040 ← positive, passes through

Layer 1, neuron #23: pre = 0.91×0.003 + 2.03×(−0.008) + (−0.01) = 0.00273 − 0.01624 − 0.01 = −0.023
h^1_{23} = ReLU(−0.023) = 0.000 ← negative, killed by ReLU

→ After Layer 1: $h^1_i \in \mathbb{R}^{128}$, roughly half are 0 (killed), half are positive features of point location

Layers 2 and 3 similarly compute: $h^3_i \in \mathbb{R}^{64}$ — a 64-number "feature vector" for this one point.

Step 2: Max-Pool Over All 200 Points → The Embedding

```
After computing h³ᵢ ∈ ℝ⁶⁴ for EACH of K=200 points:

Embedding e = max over i of h³ᵢ,  element-wise:
  e[j] = max(h³₁[j], h³₂[j], ..., h³₂₀₀[j])    for j = 1, 2, ..., 64

Each of the 64 output dimensions takes the MAXIMUM value of that feature across all 2

Result: e ∈ ℝ⁶⁴   ← the obstacle shape embedding
```

The Physical Meaning of Max-Pool Features

Imagine feature dimension j = 15 encodes "pointiness in the upper-right direction."

The MLP computes how much each individual point contributes to "pointiness in upper-right."
The max-pool picks the point that is MOST pointed in that direction across the whole cloud.
So e[15] tells us: "what's the maximum degree of upper-right pointiness in this obstacle?"

The star obstacle has 5 sharp points → e[15] would be large (some point is very pointy).
The blob obstacle has no sharp points → e[15] would be small (no point is pointy there).
The crescent has 2 horn tips → e[15] is large for the horn facing upper-right.

This is what PointNet learns: each dimension of the embedding captures the MOST EXTREME example of some learned geometric feature across the entire shape.

Why Max-Pool is Permutation-Invariant (formal proof)

```
Let σ be any permutation of {1,...,200}:
  max(h³_{σ(1)}, h³_{σ(2)}, ..., h³_{σ(200)}) = max(h³₁, h³₂, ..., h³₂₀₀)

Proof: max() doesn't care about order — it returns the largest value regardless.
(Similarly: sum, average are permutation-invariant; sorted list is NOT)
```

```
Therefore: the embedding e is IDENTICAL whether we list the points as p₁,...,p₂₀₀
or as p₁₃,p₇,p₁₄₁,...,p₅₅ — any ordering gives the same e.

This means: even if the point cloud is stored in a different order each time
(e.g., from a lidar sensor that returns points randomly), the encoder gives
the same embedding. No need to sort or register the cloud to a canonical order.
```

Side note: There is NO K-means clustering in this architecture. K-means is a different technique. In PointNet, the "K" in "K points" just means the number of points in the cloud. Max-pool replaces K-means — it's simpler and more effective.

Part B: ConditionalObstacleCBF — The Per-Obstacle Barrier Value

```
Input construction for obstacle i:
  x_rel = x − center_i          ← needle position RELATIVE to obstacle i's centroid,
  x_norm = x_rel / σ            ← normalized by the data's std (σ ∈ ℝ³, computed from
  e_i    = ObstacleEncoder(P_i) ← shape embedding ∈ ℝ⁶⁴

  network_input = concat([x_norm, e_i]) ∈ ℝ⁶⁷   (3 + 64 = 67 numbers)

Why relative position? Because the CBF cares about WHERE the needle is relative to
THIS obstacle, not where it is in the world frame.
Why normalized? Same reason as normalizing x: keeps all 67 inputs on similar scales.
```

```
Network layers (ConditionalObstacleCBF):
  dims = [67, 256, 256, 256, 1]   (input → 3 hidden layers → output)

  Layer 1: Linear(67  → 256) → Tanh   output: ℝ²⁵⁶
  Layer 2: Linear(256 → 256) → Tanh   output: ℝ²⁵⁶
  Layer 3: Linear(256 → 256) → Tanh   output: ℝ²⁵⁶
  Layer 4: Linear(256 → 1)            output: ℝ¹   [NO activation — b_i can be any si

Total parameters: (67×256+256)+(256×256+256)×2+(256×1+1)
               = 17,408 + 65,792×2 + 257 = 149,249

Last layer initialization:
  W₄ ∈ Uniform(−0.01, +0.01),  b₄ = 0
  → at start of training, b_i(x) ≈ 0 for ALL x (near-zero output)
  → gradients are well-defined from step 1 (no early saturation)
```

> **What do the 256 neurons in each hidden layer compute?**
>
> Layer 1 takes [x_rel/σ, embedding] $\in \mathbb{R}^{67} \to \mathbb{R}^{256}$.
>
> Neuron k in layer 1: out_k = Tanh($w_k^T \cdot$inp + b_k) where $w_k \in \mathbb{R}^{67}$

- Some neurons learn to detect "am I to the upper-left of THIS obstacle?" by weighting x_rel heavily
- Some neurons learn "is this obstacle star-shaped?" by weighting the embedding's "pointiness" features
- Some learn "am I close AND this is a crescent?" — combining position AND shape

After 3 layers: each neuron can detect arbitrarily complex conjunctions of position features AND shape features. The final linear layer aggregates these into a single scalar b_i(x).

What b_i(x) means physically

```
b_i(x) > 0  →  needle is safely OUTSIDE obstacle i (and in the inflation shell exteri
b_i(x) = 0  →  needle is on the learned "boundary" (≈10mm outside true tissue surface
b_i(x) < 0  →  needle is inside the inflated keep-out zone (DANGER!)
```

Part C: Smooth-Min Fusion Over N=5 Obstacles

```
After computing b₁, b₂, b₃, b₄, b₅ ∈ ℝ for all 5 obstacles:

B(x) = −(1/β) · log( exp(−β·b₁) + exp(−β·b₂) + exp(−β·b₃) + exp(−β·b₄) + exp(−β·b₅) )

where β = 1000

Equivalently (numerically stable form used in code):
  b_stack = [b₁, b₂, b₃, b₄, b₅]          ← shape (5,)
  B = −(1/1000) · logsumexp(−1000 · b_stack)  ← PyTorch's stable logsumexp
```

Why does this approximate the minimum?

Mathematical identity: $-(1/\beta)\cdot\log \Sigma_i \exp(-\beta\cdot b_i) \to \min_i(b_i)$ as $\beta \to \infty$

For β=1000: if b_1=0.015, b_2=0.003, b_3=0.040:

$\exp(-1000\times0.003) = e^{-3} \approx 0.0498$ (b_2 is smallest → largest $\exp(-\beta\cdot b)$)

$\exp(-1000\times0.015) = e^{-15} \approx 3\times10^{-7}$ (negligible)

$\exp(-1000\times0.040) = e^{-40} \approx 4\times10^{-18}$ (essentially 0)

Sum ≈ 0.0498 (b_2 dominates entirely)

$B = -(1/1000)\cdot\log(0.0498) = -(1/1000)\times(-3.00) = 0.003$ ✓

→ B ≈ min(0.015, 0.003, 0.040) = 0.003. Correct!

Why is B(x) the MINIMUM of all b_i?

> **Because safety requires being safe from ALL obstacles simultaneously.**
>
> B(x) ≥ 0 must mean "safe from ALL obstacles."
> If B(x) = min(b_i), then B≥0 iff every b_i≥0 iff safe from every obstacle. ✓
> If ANY single b_i < 0 (inside obstacle i), then min → negative → B < 0 → unsafe. ✓
>
> The smooth-min approximates this exactly for β=1000 (error < ln(5)/1000 ≈ 1.6mm, which is below the 3mm analytic safety margin — practically negligible).

Gradient ∇B_φ(x): The Direction to Safety

```
Code: model.B.gradient(x_tensor)

Implementation:
  xr = x.detach().requires_grad_(True)   ← make PyTorch track gradients wrt x
  B  = model.B.forward(xr)               ← full forward: encoder → CBF → smooth-min
  grad = autograd.grad(B.sum(), xr)[0]   ← backprop all the way to x

Result: gradB ∈ ℝ³   ← points in the direction that INCREASES B

Physical meaning:
  gradB points AWAY from the nearest obstacle (roughly).
  Moving in direction +gradB → moving to a safer position.
  Moving in direction −gradB → moving toward the obstacle (DANGEROUS).
```

PART II — RESNET: What It Is and Where It's Actually Used

II.1 ResNet Block Explained

The previous PDF mentioned ResNet for f_θ — this was INCORRECT. f_θ uses a plain MLP. ResNet is mentioned in CLAUDE.md as a potential architecture, but the actual implemented f_θ is a plain 4-layer MLP. Let me explain what ResNet would be, since you asked:

```
Plain MLP block:
  out = Tanh(W · x + b)

ResNet block (skip connection):
  z   = Tanh(W₂ · Tanh(W₁ · x + b₁) + b₂)
  out = z + x      ← add the ORIGINAL INPUT back

The key: the output is z + x, not just z.
The block only needs to learn z = out − x (the "residual").

Gradient of loss through a ResNet block:
  d(loss)/d(x) = d(loss)/d(out) · d(out)/d(x)
               = d(loss)/d(out) · (d(z)/d(x) + 1)   ← the "+1" from the skip!
Even if d(z)/d(x) ≈ 0 (vanishing), the gradient is still d(loss)/d(out) · 1 → no vani
```

In OUR project, ResNet blocks are NOT used. The PointNet's per-point MLP uses ReLU (which also avoids vanishing gradients) and the CBF MLP uses Tanh with depth 3 (not deep enough to need ResNet).

PART III — COMPLETE TRAINING PIPELINE

III.1 Overview: Two-Stage Training

```
STAGE 1 (train.py, ~2.5 hours on GPU):
  Phase 0: Pretrain f_θ ALONE on imitation loss (300 epochs, RK4 rollout MSE)
  Outer loop (OUTER_ITERS=10 iterations):
    Inner loop (INNER_EPOCHS=150 epochs per outer iteration):
      Jointly train f_θ + V_θ + B_φ on composite_barrier_loss
      (barrier weight RAMPS from 0 → full over outer iters 0..2)
    After inner loop: sample N_CEX=2000 counter-examples (CEX)
    Add violations to training sets
    If outer_iter ≥ 3 and no violations: converged → save

STAGE 2 (train_barrier_fast.py, ~3-5 minutes on GPU):
  Load trained f_θ and V_θ from Stage 1, FREEZE them
  Re-initialize B_φ (reset weights — fresh barrier training)
  Train ONLY B_φ for 12,000 steps on augmented_barrier_loss
  → Teaches B_φ to generalize to arbitrary obstacle poses/scales
```

III.2 Phase 0: Pretraining f_θ with Neural ODE Loss

```
For each demo trajectory {x(t₀), x(t₁), ..., x(t_T)}:

  1. Start from x(t₀) = x_start
  2. Integrate ẋ = f_θ(x, s) using RK4 with dt=0.02s for T steps → predicted trajecto

RK4 step (ONE step from x̂ₖ to x̂ₖ₊₁):
  k₁ = f_θ(x̂ₖ,              s(x̂ₖ))
  k₂ = f_θ(x̂ₖ + dt/2·k₁,   s(x̂ₖ+dt/2·k₁))
  k₃ = f_θ(x̂ₖ + dt/2·k₂,   s(x̂ₖ+dt/2·k₂))
  k₄ = f_θ(x̂ₖ + dt·k₃,     s(x̂ₖ+dt·k₃))
  x̂ₖ₊₁ = x̂ₖ + (dt/6)·(k₁ + 2k₂ + 2k₃ + k₄)

Loss: L_MSE = (1/M·T) Σᵢ Σₖ ‖x̂ᵢ(tₖ) − xᵢ(tₖ)‖²   (mean over demos × timesteps)

After 300 epochs: f_θ can reproduce all demo trajectories by integration.
Check: run f_θ from x_start, watch it arrive near x_goal.
```

III.3 Stage 1 Inner Loop: The Composite Barrier Loss

At each training step, we sample points from 3 regions:

```
Region 1 — Safe set X_safe: points with sdf(x) > INFLATE + 0.004 = 0.014m
  (far outside the inflation shell — definitely safe)
  → Sample from workspace minus the keep-out zones

Region 2 — Unsafe set X_unsafe: points with sdf(x) < INFLATE = 0.010m
  (inside the obstacle OR in the 10mm inflation shell — danger zone)
  → Sample from canonical interior cloud (transformed) + random points near obstacle

Region 3 — Demo path S: points on/near the demo path
  → Sample demo waypoints + small perturbations
```

```
LOSS TERM 1: L_safe — SDF REGRESSION on safe set
  Target: b_target(x) = K · clip(sdf(x) − INFLATE, −CLAMP_IN, +CLAMP_OUT)
         K = 4.0, INFLATE = 0.010, CLAMP_IN = 0.040, CLAMP_OUT = 0.013

  For x with sdf = 0.025m: b_target = 4·clip(0.025−0.010, −0.040, +0.013)
                                    = 4·clip(0.015, ...)  = 4·0.013 = 0.052  [clampe
  For x with sdf = 0.012m: b_target = 4·clip(0.002, ...)  = 4·0.002 = 0.008
  For x with sdf = 0.005m: b_target = 4·clip(−0.005, ...) = 4·(−0.005) = −0.020

  L_safe = (1/N_safe) Σ (B_φ(x) − b_target(x))²    [MSE, forces B to match the SDF sh

  Weight: λ_FS = 5.0
```

```
LOSS TERM 2: L_unsafe — HINGE LOSS on unsafe set
  "The network must predict B < 0 for all points in/near the obstacle"

  L_unsafe = (1/N_unsafe) Σ max(0, B_φ(x) + 0.001)²

  If B_φ(x) = −0.005 (correctly negative): max(0, −0.005+0.001) = max(0,−0.004) = 0 →
  If B_φ(x) = +0.010 (WRONG, positive inside): max(0, 0.010+0.001) = 0.011 → penalize

  Weight: λ_OB = 12.0   (high, because safety violations are critical)
```

```
LOSS TERM 3: L_dot — CLF DECREASE CONDITION
  "The demo flow f_θ must make V decrease over time"

  V̇ = ∇V · f_θ(x, s)   should be ≤ −α·V

  L_dot = (1/N) Σ max(0, ∇V·f_θ(x,s) + α·V(e))²   [hinge: 0 if satisfied, positive if
  α = ALPHA_CLF = 4.0,   Weight: λ_DOT = 0.5

  + L_V1 = (1/N_safe) Σ max(0, DELTA_V − V(e))²   [V > δ_V far from path]
  + L_V2 = (1/N_demo) Σ (V(e))²                    [V ≈ 0 on demo path]
```

```
TOTAL Stage 1 loss:
  L = λ_MSE·L_MSE + λ_V1·L_V1 + λ_V2·L_V2 + λ_FS·L_safe + λ_OB·L_unsafe + λ_DOT·L_dot
  (with ramp: barrier terms multiplied by min(outer_iter/3, 1) for outer_iters 0,1,2)
```

III.4 Stage 1: Counter-Example (CEX) Refinement

```
After each inner loop (150 epochs), sample N_CEX=2000 random points from the workspac
For each sampled point x:
  Compute B_φ(x), V(x), f_θ(x)
  Check Lyapunov: is V(e) ≤ 0 when it should be > 0?
  Check barrier: is B_φ(x) > B_SAFE_MARGIN=0.008 but sdf(x) < INFLATE? (network says

Any violation: add x to the CEX buffer (N_CEX=2000 stored violations)
Next inner loop: sample 20% of each batch from CEX buffer → force the network to fix

This is "counterexample-guided training" from S2-NNDS Algorithm 2.
```

III.5 Stage 2: Augmented Barrier Loss — The Key to Pose Generalization

What is augmentation and why does it work?

```
Before each training step (augment_scene):
  For each obstacle i, draw:
    θ   ∈ Uniform(−π, +π)   ← rotation angle (can be ANY direction!)
    σ   ∈ Uniform(0.65, 1.4) ← scale (65% to 140% of original size)
    t   ∈ Uniform(−0.05m, +0.05m)² ← translation (±5cm)

  Rotation matrix: R = [[cos θ, −sin θ], [sin θ, cos θ]]

  Apply to CLOUD: p_aug = σ·R·p_original + t   (for each of the 200 cloud points)
  Apply to SDF:   sdf_aug(x) = σ · sdf_canonical(R^{−1}·(x−center−t) / σ)
                  (exact formula — SDF of a transformed shape is the transform applie

  The SAME (θ, σ, t) is applied to BOTH the cloud AND the SDF labels.
  Therefore: the network sees a rotated star cloud AND gets the correct SDF labels fo
  After 12,000 steps: the network has seen the star at ~12,000 different poses → gene
```

Why freeze f and V in Stage 2?

> **Three reasons to freeze f and V:**
>
> 1. **f doesn't depend on obstacle pose.** The demo trajectory is fixed — rotating an obstacle doesn't change where the demo path goes. There's nothing about obstacle augmentation that should change f's weights. If we let

f train with augmentation, random gradients from obstacle samples would corrupt the demo-following behavior we worked hard to get in Stage 1.

2. **V doesn't depend on obstacle pose either.** Lyapunov stability is about convergence to the demo path — it doesn't know or care about obstacles. Same argument: augmentation gradients would only corrupt the Lyapunov conditions.

3. **Speed.** Freezing f (34k params) and V (4.5k params) means only B's 149k params are updated → 3× fewer computations per gradient step → Stage 2 finishes in 3-5 minutes instead of hours.

III.6 The Eikonal Term — Why ‖∇B‖ Must Be Controlled

What went wrong without it

The "near-step function" failure (real bug from development):

The hinge loss L_unsafe only says "B must be negative inside." It does NOT say HOW negative or HOW steeply B must change. The network discovered a shortcut: learn B ≈ +1 everywhere outside and B ≈ −1 everywhere inside, with an EXTREMELY thin transition (less than 1mm wide).

Effect: ‖∇B‖ ≈ 290 m^{-1} at the boundary (near-step function, not an SDF).

Now in the QP: cbf_rhs = −γ(B−margin) − ∇B·f_eff
With ‖∇B‖=290 and f_eff pointing toward the obstacle (0.04 m/s along −∇B direction):
∇B·f_eff = 290 × (−0.04) × cos(angle) ≈ −11.6 (large negative!)
cbf_rhs = −3×(0.018−0.008) − (−11.6) = −0.030 + 11.6 = +11.57

The CBF constraint became: ∇B·u ≥ 11.57 → u must push along ∇B with magnitude 11.57/290 ≈ 0.04 m/s
But sometimes the angle was slightly different and cbf_rhs became NEGATIVE → "no push needed" → u=0 → needle carried straight through by the K=5 demo spring.

The eikonal term — making ‖∇B‖ well-behaved

```
L_eikonal = λ_EIK · (1/N) Σ ReLU(‖∇B_φ(x)‖ − B_GRAD_MAX)²

where B_GRAD_MAX = 12.0,  λ_EIK = 0.02

This term is 0 when ‖∇B‖ ≤ 12  (no penalty — gradient is fine)
This term is positive when ‖∇B‖ > 12  (penalty — gradient is too large, push it down)

Combined with L_safe (regression to SDF target with K=4):
  L_safe trains: B ≈ K·clip(sdf−Δ, ...) = 4·(sdf−0.010) in the exterior
```

```
   This means ∇B ≈ K·∇sdf = 4·(unit outward normal) → ‖∇B‖ ≈ 4 in the exterior
   L_eikonal caps at 12 (3×K) to handle boundary artifacts without blocking the K=4 ta

 Result: ‖∇B‖ ≈ 4-12 throughout → QP is well-conditioned → no more step-function failu
```

> **Physical interpretation of the eikonal condition:**
> In a true SDF: ‖∇SDF‖ = 1 everywhere (exactly 1 m per meter of distance). In our learned barrier: ‖∇B‖ ≈ K = 4 (we scale the SDF by K=4 for sharper gradients in the QP). The eikonal term makes the barrier behave like a "smooth scaled SDF" — gradual, predictable, with a constant slope. This is what makes the CBF QP work reliably.

III.7 Training Parameter Summary

Parameter	Value	Where used
PHASE0_EPOCHS	300	f_θ pre-training epochs
OUTER_ITERS	10	CEX refinement outer loop
INNER_EPOCHS	150	Gradient steps per outer iteration
N_CEX	2000	Counter-examples stored per outer iter
LAMBDA_MSE	50.0	Weight on imitation rollout loss
LAMBDA_B_FS	5.0	Weight on safe-set SDF regression
LAMBDA_B_OB	12.0	Weight on unsafe hinge loss (strict!)
LAMBDA_B_DOT	0.5	Weight on CBF decrease condition
LAMBDA_B_REG	20.0	Weight on SDF regression (Stage 2)
LAMBDA_EIK	0.02	Eikonal gradient cap penalty
B_GRAD_MAX	12.0	Max allowed ‖∇B‖ before eikonal penalizes
AUG_ROT_RANGE	(−π, +π)	Stage 2 rotation augmentation range
AUG_SCALE_RANGE	(0.65, 1.4)	Stage 2 scale augmentation range
AUG_TRANS_RANGE	±0.05 m	Stage 2 translation augmentation range
Stage 2 steps	12,000	Barrier-only fast training steps

PART IV — ONLINE INFERENCE: THE COMPLETE QP PIPELINE

IV.1 What Happens Every 10ms (100 Hz)

Every control cycle runs the following 10 steps. Here we walk through each one completely with concrete numbers from a near-obstacle scenario.

```
SCENARIO: Needle approaching the star obstacle head-on.
  x = [0.498, 0.062, 0.440] m     ← very close to star center (0.500, 0.050, 0.440)
  star center c = [0.500, 0.050, 0.440]
  Approximate SDF to star ≈ 0.012 m (12mm outside star's surface, but inside inflatio
  B_num ≈ 0.018  (learned barrier)
  The demo flow points roughly toward x_goal (which is PAST the star) → potential con
```

Step 1: Get s (progress)

```
x = [0.498, 0.062, 0.440]
Find nearest demo waypoint:
  demo[6] = [0.512, 0.091, 0.440],  ‖x − demo[6]‖ = ‖[−0.014, −0.029, 0]‖ = 0.032m
  demo[5] = [0.500, 0.091, 0.440],  ‖x − demo[5]‖ = ‖[−0.002, −0.029, 0]‖ = 0.029m ←
idx = 5,  s = 5/9 = 0.556
```

Step 2: Neural Forward Pass → f_val

```
x̃ = (x − μ) / σ = ([0.498,0.062,0.440] − [0.490,0.080,0.440]) / [0.028,0.018,0.003]
  = [0.286, −1.000, 0.0]
inp = [0.286, −1.000, 0.0, 0.556] ∈ ℝ⁴

→ Through 4 MLP layers → v_ff = [0.025, 0.030, 0.0] m/s   (network says: go forward a

x_ref(0.556) = interpolated demo = [0.503, 0.091, 0.440]
v_fb = 5 × ([0.503,0.091,0.440] − [0.498,0.062,0.440]) = 5×[0.005, 0.029, 0] = [0.025

f_val = v_ff + v_fb = [0.025+0.025, 0.030+0.145, 0] = [0.050, 0.175, 0.0] m/s
  ← Strong pull UPWARD (toward demo path which is at y=0.091) and forward
```

Step 3: Barrier Forward Pass → B_num and gradB

```
For star obstacle (center = [0.500, 0.050, 0.440], cloud P_star ∈ ℝ^(200×2)):

  e_star = ObstacleEncoder(P_star) → embedding ∈ ℝ⁶⁴

  x_rel = [0.498,0.062,0.440] − [0.500,0.050,0.440] = [−0.002, 0.012, 0.000]
```

```
  x_norm = x_rel / σ = [−0.002/0.028, 0.012/0.018, 0] = [−0.071, 0.667, 0]
  input_cbf = [−0.071, 0.667, 0.0, e_star₁, ..., e_star₆₄] ∈ ℝ⁶⁷

  b_star = ConditionalObstacleCBF(input_cbf) → (scalar) e.g. 0.018

Similarly compute b_crescent, b_blob, b_kidney, b_lshape ≈ [0.041, 0.055, 0.067, 0.07
(other obstacles are farther away)

Smooth-min:
  exp(−1000×0.018) = e⁻¹⁸ ≈ 1.5×10⁻⁸
  exp(−1000×0.041) = e⁻⁴¹ ≈ 2.1×10⁻¹⁸  (essentially 0)
  Sum ≈ 1.5×10⁻⁸  (star obstacle completely dominates)
  B = −(1/1000)·log(1.5×10⁻⁸) ≈ −(1/1000)×(−18) = 0.018 m

autograd → gradB = [−0.85, 0.50, 0.0]  (points away from star: left and slightly up)
```

Step 4: CLF Terms

```
e = x − x_ref(s) = [0.498,0.062,0.440] − [0.503,0.091,0.440] = [−0.005, −0.029, 0]
V = ‖e‖² = (−0.005)² + (−0.029)² = 0.000025 + 0.000841 = 0.000866
gradV = 2e = [−0.010, −0.058, 0.0]
```

Step 5: Go-Around Guidance (Swirl)

```
Check: B_num = 0.018 < B_ACTIVE = 0.040 → YES, apply swirl

n = gradB / ‖gradB‖ = [−0.85, 0.50, 0] / 0.986 = [−0.862, 0.507, 0]
t = [−n₂, n₁, 0] = [−0.507, −0.862, 0]   ← tangent (perpendicular to n in XY)

goal_direction = (x_goal − x) / ‖...‖ = ([0.548,0.063,0.44]−[0.498,0.062,0.44]) / ‖..
               = [0.050, 0.001, 0] / 0.050 = [1.000, 0.020, 0]

dot(t, goal_dir) = [−0.507,−0.862,0]·[1.000,0.020,0] = −0.507 − 0.017 = −0.524  < 0
→ FLIP t:  t = [0.507, 0.862, 0]   ← now points to the right and up (toward goal)

closeness = max(0, (0.040 − 0.018) / 0.040) = 0.022/0.040 = 0.55
SWIRL_GAIN = 1.0,  ‖f_val‖ = ‖[0.050, 0.175, 0]‖ = 0.182 m/s

g = 1.0 × 0.182 × 0.55 × [0.507, 0.862, 0] = [0.051, 0.086, 0] m/s

f_eff = f_val + g = [0.050+0.051, 0.175+0.086, 0] = [0.101, 0.261, 0.0] m/s

Note: g·gradB = [0.051,0.086,0]·[−0.85,0.50,0] = −0.043+0.043 ≈ 0  ✓ (tangential, doe
```

Step 6: Compute QP Right-Hand Sides

```
CBF RHS:
  cbf_rhs = −γ·(B_num − margin) − gradB·f_eff
          = −3.0×(0.018 − 0.008) − [−0.85, 0.50, 0]·[0.101, 0.261, 0]
          = −3.0×0.010 − (−0.0859 + 0.1305)
          = −0.030 − 0.0446
          = −0.075

(Negative cbf_rhs: f_eff is helping the CBF — pushing away from the obstacle —
 so we might not even need a correction u!)

CLF RHS:
  clf_rhs = −α·V − gradV·f_eff
          = −4.0×0.000866 − [−0.010, −0.058, 0]·[0.101, 0.261, 0]
          = −0.003464 − (−0.00101 − 0.01514)
          = −0.003464 + 0.01615
          = +0.01269

(Positive clf_rhs: the CLF already decreases fast enough with f_eff alone)
```

Step 7: Build the OSQP Problem

```
Decision variable: z = [u₁, u₂, u₃, ε] ∈ ℝ⁴   (3D velocity correction + slack)

Objective matrix P (4×4 diagonal):
  P = diag(2, 2, 2, 2×λ) = diag(2, 2, 2, 1.0)   [λ=0.5 → 2λ=1]

  Minimizes: z^T P z / 2 = u₁² + u₂² + u₃² + 0.5ε²

q = [0, 0, 0, 0]   (no linear term)

Constraint matrix A (3×4):
  CBF row:  [gradB₁, gradB₂, gradB₃,  0  ] = [−0.85, 0.50, 0.0, 0.0]
  CLF row:  [gradV₁, gradV₂, gradV₃, −1.0] = [−0.010, −0.058, 0.0, −1.0]
  ε≥0 row:  [0,      0,      0,       1.0] = [0.0, 0.0, 0.0, 1.0]

Lower bounds l: [cbf_rhs, −∞, 0] = [−0.075, −∞, 0]
Upper bounds u: [+∞, clf_rhs, +∞] = [+∞, +0.01269, +∞]

Constraints:
  CBF: −0.85u₁ + 0.50u₂ ≥ −0.075   (must push outward enough)
  CLF: −0.010u₁ − 0.058u₂ − ε ≤ +0.01269
  ε ≥ 0
```

Step 8: OSQP Solves the QP

```
OSQP iterates the ADMM algorithm:
  Each iteration: split z into primal/dual parts, update alternately
  Convergence in ~20 iterations (warm start from previous step)
```

```
Solution (approximate, since cbf_rhs=-0.075 means f_eff already safe):
  u ≈ [0, 0, 0],  ε ≈ 0

Verify CBF: −0.85×0 + 0.50×0 = 0 ≥ −0.075 ✓
Verify CLF: −0.010×0 − 0.058×0 − 0 = 0 ≤ +0.01269 ✓
→ QP solver says: no additional correction needed — the swirl guidance handles it!

u_qp = [0, 0, 0],  slack = 0
```

Step 9: Combine and project_safe

```
u_safe = g + u_qp = [0.051, 0.086, 0] + [0, 0, 0] = [0.051, 0.086, 0]

v_total = f_val + u_safe = [0.050+0.051, 0.175+0.086, 0] = [0.101, 0.261, 0.0] m/s

project_safe check:
  x_next = x + dt × v_total = [0.498,0.062,0.44] + 0.01×[0.101,0.261,0] = [0.4990, 0.
  B(x_next) via network ≈ 0.021  (moved slightly away from star center x₁=0.500)
  thresh = min(0.018, 0.008) − 1e-6 ≈ 0.008
  0.021 ≥ 0.008 ✓ → no projection needed, keep v_total
```

Step 10: Analytic Safety Filter (Final Check)

```
sdf_min(x_next) using EXACT analytic star SDF:
  x_next relative to star center: [−0.001, +0.0146, 0] → sdf ≈ 0.015m
  Compare to SAFETY_SDF_MARGIN = 0.011m: 0.015 ≥ 0.011 ✓ → ACCEPT

Final output: v_safe = [0.101, 0.261, 0.0] m/s
→ Send to robot: move forward at 10 cm/s AND up at 26 cm/s (going around the star)
→ Next step: x will be at [0.499, 0.065, 0.440] — moving away from direct approach, c
```

PART V — LITERATURE MAP: What Each Paper Contributed

V.1 Primary Papers Used

Paper (in /docs)	What We Took From It
Learning Complex Motion Plans using Neural ODEs with Safety and Stability Guarantees (Nawaz et al. 2023 — cbf_paper.pdf or Learning Complex...pdf)	**The core architecture template.** • ProgressConditionedDS f_θ concept: demo path as attractor, progress s as input • Quadratic CLF: $V(e) = \|e\|^2$ (the base formula we use) • CLF-CBF QP formulation (Eq. 14): min $\|u\|^2+\lambda\varepsilon^2$ s.t. CBF hard + CLF soft • QP parameters: $\alpha=4$, $\gamma=3$, $\lambda=0.5$ (directly copied from their paper) • ODE training: RK4 rollout MSE as imitation loss (L_{MSE} = Nawaz Eq. 4) • Algorithm structure: pretrain f, then joint CLF+CBF training *We extended it by: replacing analytic sphere-CBF with learned CompositeBarrier, adding V correction term, adding pose augmentation*
CN-CBF: Composite Neural Control Barrier Function for Safe Robot Navigation in Dynamic Environments	**The composite barrier architecture.** • Smooth-min formula: $B = -(1/\beta)\cdot\log \Sigma_i \exp(-\beta\cdot b_i)$ — Equation 18 in CN-CBF paper • PointNet-style permutation-invariant encoder concept • Per-obstacle conditional CBF: $b_i(x, e_i)$ shared network with embedding • $\beta=1000$ value choice and its relationship to error bound $\ln(N)/\beta$ • Multi-obstacle composition: handles arbitrary number of obstacles • Dynamic obstacle handling: re-evaluate each step with updated obstacle info *We added: pose/scale augmentation,*

	SDF-regression training target, Stage 2 fast retraining
Safe and Stable Neural Network Dynamical Systems for Robot Motion Planning (S2-NNDS)	**Training algorithm (Algorithm 2) and jointly training f/V/B.** • Algorithm 2: joint training of f_θ, V_θ, B_φ with CEX refinement • Counter-example (CEX) buffer: add violations to training set, 20% mini-batch from CEX • Sign convention for B: B>δ safe, B<−δ unsafe (our δ=B_SAFE_MARGIN=0.008) • Fully learned barrier (no analytic SDF anchor) — the BarrierNet concept • Simultaneous imitation + Lyapunov + barrier training • project_safe style: discrete-time barrier projection after continuous QP *We added: CompositeBarrier (vs single BarrierNet), augmentation, eikonal term*
Learning_Control_Barrier_Functions_from_Expert_Demonstrations	**Motivation and training methodology for CBF from data.** • Using SDF labels as supervised training targets for the learned barrier • The idea that demo trajectories implicitly define a "safe corridor" • Hinge loss formulation for unsafe points (L_unsafe) • Counter-example refinement concept
Sequential Neural Barriers for Scalable Dynamic Obstacle Avoidance	**Composition ideas and dynamic obstacle handling.** • Idea of composing multiple barrier certificates over many obstacles • Dynamic environment: re-evaluating the barrier at each control step • Scalability via shared network weights across obstacles *We implemented the sharing differently: via embedding conditioning instead of sequential composition*

A Survey on CLF and CBF for Nonlinear-Affine Control Systems	**Theoretical foundation.** • Formal CBF definition and validity conditions (class-K functions) • CLF stability proofs and Lyapunov-based control • Combined CLF-CBF QP and feasibility conditions • The constraint: $\nabla B \cdot (f+u) \geq -\gamma \cdot B$ (class-K function with γ linear → exponential CBF) • Why γ controls aggressiveness of safety enforcement
Learning_Robust_Output_Control_Barrier_Functions_from_Safe_Expert	**Robustness ideas.** • Training barriers to be robust to modeling errors • Margin/buffer in safe set (our B_SAFE_MARGIN = 0.008 as buffer) • Handling of output-feedback vs state-feedback
Beik-Mohammadi: Extended Neural Contractive Dynamical Systems	**Demo flow architecture ideas.** • Progress-conditioned velocity fields • Riemannian safety regions concept • Demo interpolation (x_ref(s)) and feedback correction $K \cdot (x_{ref} - x)$ • Multiple tasks conditioned on the same network structure
BP_CBF_Technical_Pitch.pdf	**The original project concept.** • The tolerance/penetration budget idea (η_ψ — later removed per mentor feedback) • Initial framing of the needle insertion safety problem • The hybrid learned+analytic safety filter idea • Early architecture sketches

V.2 Papers Consulted but Not Directly Used

Paper	Why Consulted
HJ Reachability papers (several)	Alternative to CBF for safety; decided CBF-QP was simpler and real-time feasible

RAIL: Reachability-Aided Imitation Learning	Inspiration for combining imitation learning with safety guarantees
Online Learning-Enhanced High Order Adaptive Safety Control	Adaptive/online updates — not implemented but informed Stage 2 fast retraining idea
V-OCBF: Learning Safety Filters from Offline Data	Offline-learned safety filter — alternative to our hybrid approach
A Collision Cone Approach for CBF	Velocity-obstacle style CBF — considered for moving obstacles

PART VI — HONEST ICRA ASSESSMENT

VI.1 What ICRA 2026 Requires

ICRA (IEEE International Conference on Robotics and Automation) is the top robotics venue. Acceptance rate ≈ 40%. Strong papers have: (1) a clear novel contribution, (2) validated on real hardware OR very strong simulation study with many baselines, (3) theoretical analysis, (4) comparison to prior work.

VI.2 Current Strengths

What is genuinely novel and well-executed:

1. **Pose-generalizing composite learned CBF**: Combining PointNet encoding + smooth-min composition + pose augmentation to get zero-shot generalization is a clean contribution. CN-CBF paper showed the composition, S2-NNDS showed learned CBF — combining them with augmentation for pose invariance is NEW.
2. **Two-stage training strategy**: Freeze f/V after Stage 1, fast-retrain only B with augmentation. Practical and works. The barrier-only 3-minute retraining is deployable.
3. **Hybrid safety filter**: Learned CBF drives the field, exact SDF provides geometric backstop. The "learn the hard stuff, check the easy stuff" decomposition is principled.
4. **Worked on non-convex irregular shapes**: star/crescent/kidney are hard. Most CBF papers use circles or convex polytopes. Your shapes are harder.
5. **SDF-shaped barrier target with asymmetric clamping**: The $K \cdot \text{clip}(\text{sdf} - \Delta, -\text{CLAMP\_IN}, +\text{CLAMP\_OUT})$ target with CLAMP_IN>CLAMP_OUT is a subtle but important design choice (keeps ∇B persistent through the interior).

VI.3 Current Weaknesses (Honest Assessment)

What would likely be rejected by reviewers:

1. **No hardware experiments.** The biggest problem. ICRA reviewers expect results on the actual FR3 robot. Simulation-only results are insufficient for a systems/control paper. The paper must show: real needle on real foam, real obstacles, real penetration counts.
2. **2D task in a 3D world.** The needle is at fixed Z=0.44m — it's effectively a 2D problem. Reviewers will ask: does this extend to 3D needle insertion? If not, why is this paper about robotics?
3. **No baselines.** To claim novelty of the pose-generalizing CBF, you need to compare against: (a) CBF trained at single pose (fails at new poses), (b) CBF retrained at each new pose (works but slow), (c) SDF-based CBF directly (works if SDF known). Current paper shows only your method.

4. **"Hybrid safety" is somewhat ad-hoc.** The analytic_safety_filter essentially bypasses the learned barrier when it matters most. Reviewers may ask: "if you have the exact SDF, why not just use it everywhere?" You need a clear answer (e.g., the exact SDF isn't available at deployment, or the learned field provides better trajectories).
5. **Barrier "compression" problem.** You documented it in the memory: the learned B has slope ~1.5 instead of K=4, giving effective standoff ~25-40mm instead of the 10mm design. This means the approach is more conservative than claimed, and you don't fully understand why or how to fix it.
6. **No formal guarantees.** CBF papers ideally prove that the closed-loop system maintains $B \geq 0$ with probability 1, or under certain assumptions. Current work only provides empirical results (obstacle=0 in testing). The analytic filter provides the formal guarantee, but then the learned part is just a heuristic.
7. **Limited evaluation.** 5 obstacle types, 8 configurations, 4 dynamic motions. ICRA papers typically have much larger evaluation (50+ configurations, statistical analysis with error bars).

VI.4 What to Improve for ICRA Submission

Prioritized improvement list (do these in order of impact): MUST DO (dealbreakers without these):

1. **Hardware experiments on FR3.** Even 5-10 real needle insertions with 0 penetrations would be compelling. This requires: calibrate obstacle positions on real foam, run the controller at 100 Hz, measure penetration with force sensors or visual tracking.
2. **Add baselines:**
   - Baseline 1: "Fixed-pose CBF" — trained at canonical pose, tested at rotated poses → should fail
   - Baseline 2: "SDF-CBF" — use exact analytic SDF directly (no learning) → fails when SDF not known
   - Baseline 3: "Learning to Nudge" (the "Nudge" paper from your /docs) — dense obstacle setting
3. **Formalize the contributions.** Write: "Our contribution is X, which generalizes prior work Y in way Z." Be specific about what is new.

SHOULD DO (significantly strengthen the paper):

4. **Fix or acknowledge the barrier compression.** Either: (a) redesign training to fix it, or (b) formally account for it in the theory (if slope is α instead of K, the effective margin is $\alpha/K \times$ nominal_margin).
5. **Statistical evaluation.** Run 50+ random obstacle configurations, report: mean penetration rate, success rate (reach goal), mean deviation from demo, running time per control step. Show confidence intervals.
6. **Formal robustness bound.** Prove: "if the learned B has gradient error ε_g, the analytic filter with margin m provides safety within X mm." Even a simple informal analysis would help.

NICE TO HAVE (above and beyond):

7. Extend to 3D (requires 3D point clouds and 3D SDF)
8. Adaptive Stage 2 retraining when obstacles move (online learning)
9. Comparison on the RAIL, NUDGE, or CN-CBF benchmark environments

VI.5 Summary Verdict

Current state: NOT ready for ICRA. Needs 3-6 months of work.

The core ideas are novel and promising. The implementation works. But without hardware experiments and baselines, a top venue like ICRA would reject it. Here is a realistic path:

Month 1-2: Hardware experiments on FR3. This is the most critical thing.
Month 2-3: Implement and test 2-3 baselines. Write up clean comparison tables.
Month 3-4: Write the paper. Frame the contribution clearly. Add formal analysis section.
Month 4-5: Internal review, iterate, polish. Submit to ICRA 2027 or RA-L.

Alternatively: If hardware isn't available, submit to a workshop (ICRA Safety Workshop) as a work-in-progress, get feedback, then extend to a full journal paper (RA-L or T-RO).

PART VII — Q&A: Comprehensive Questions and Answers

Category 1: Basic / Conceptual Questions

Q1: What is the TA-CBF project trying to do in one sentence?

> A: Teach a robot needle to navigate from start to goal through foam, strictly avoiding 5 critical tissue regions of non-convex irregular shapes, even when those obstacles appear at poses and scales never seen during training — all in real time.

Q2: Why is this called "Tolerance-Aware"?

> A: The original design (BP_CBF_Technical_Pitch.pdf) included a "Penetration Budget" η_ψ that allowed limited tolerance for CBF classification error — the system could "tolerate" small barrier errors up to a budget. This tolerance-aware component was later removed per mentor feedback (exact geometric backstop replaces it), but the name persisted. Currently "tolerance" refers to the system's ability to tolerate pose variation through augmentation, and the 10mm inflation margin tolerates small calibration errors.

Q3: What is a Control Barrier Function (CBF) conceptually?

> A: A CBF is a scalar function B(x) that acts like a "force field" around obstacles. When B(x) is large (far from obstacle), it relaxes and lets the robot do what it wants. When B(x) is small (near obstacle), it "stiffens" and forces the robot's velocity to point away from the obstacle. Mathematically, it guarantees that if you start safe ($B \geq 0$), you stay safe forever — as long as the controller satisfies the CBF constraint $\dot{B} \geq -\gamma B$.

Q4: What is the role of the QP? When does it activate?

> A: The QP runs at every control step (100 Hz), but it usually produces $u \approx 0$ when the nominal flow f_θ is already safe. It "activates" meaningfully only when f_θ would violate the CBF constraint (pushing the needle toward an obstacle). Then it finds the MINIMUM correction u that makes both the CBF and CLF constraints satisfied simultaneously. You can think of it as: the demo flow is the driver, and the QP is the safety co-pilot that only intervenes when necessary.

Category 2: Technical / Architecture Questions

Q5: The ConditionalObstacleCBF takes 67-dimensional input — but that contains the obstacle embedding. Doesn't this mean the same network must recognize ALL obstacle shapes?

> A: Yes, exactly! This is the design. The SAME CBF MLP ($67 \to 256 \to 256 \to 256 \to 1$) handles all obstacle types. The obstacle's identity and shape come through the embedding $e_i \in \mathbb{R}^{64}$ (which encodes "star", "crescent",

> etc.). The relative position x_rel tells "where am I relative to this obstacle." The network learns: "given that I'm at relative position x_norm AND the obstacle has shape e_i, how far inside/outside am I?" Because the embedding is different for each obstacle type, the network can specialize its behavior per-obstacle via the 64 conditioning dimensions.

Q6: Why 200 points in the cloud? Why not 1000 or 20?

> A: Trade-off between coverage and speed. 200 points per obstacle × 5 obstacles = 1000 forward passes through the point_mlp at each control step. With 25,152 params in the encoder, this is computationally affordable at 100 Hz. 20 points would miss thin regions (crescent horns, star tips) — the max-pool would never "see" the spike geometry. 1000 points gives marginally better coverage but 5× the computation. 200 points was empirically validated to capture all 5 shapes adequately.

Q7: Why does the encoder NOT have an activation after the last per-point layer?

> A: The last per-point layer outputs $h^3\_i \in \mathbb{R}^{64}$ WITHOUT activation. This is standard PointNet design. If we applied ReLU or Tanh to h^3_i before the max-pool, negative features would be killed (ReLU) or squashed (Tanh) before comparison. Without activation, the max-pool selects the true maximum pre-activation value across all points, giving the CBF MLP the full range of feature values. The CBF MLP's first layer then applies Tanh — so non-linearity is still there, just shifted by one step.

Q8: The CLF uses gradV = 2e (a closed-form formula). Why not let the network compute gradV automatically via autograd?

> A: Because $\nabla(\|e\|^2) = 2e$ is EXACTLY known analytically. Computing it with autograd would give the same answer but cost time and potentially introduce numerical errors. The correction term $\delta \cdot corr$ has a small gradient contribution (δ=0.05 is small), and in the QP code (cbf_qp.py), the gradient of V is approximated as just 2e (the quadratic part). This approximation is valid near the demo path where the QP operates — the correction's gradient is proportional to δ and vanishes as $e \to 0$.

Q9: The smooth-min formula B = −(1/β)·logsumexp(−β·b). What is logsumexp and why use it?

> A: The naive formula exp(−1000×0.003) involves raising e to the power −3 → 0.05, which is fine. But exp(−1000×−0.05) = exp(+50) → 5.2×10^{21}, which overflows float32! logsumexp computes $\log(\Sigma \exp(x_i))$ stably by subtracting the max first: logsumexp(x) = max(x) + $\log(\Sigma \exp(x_i - \max(x)))$ Each $\exp(x_i - \max) \le 1$ → no overflow. PyTorch's torch.logsumexp uses this internally. So −(1/β)·logsumexp(−β·b) gives the smooth-min without overflow or underflow.

Category 3: Mathematical Questions

Q10: Prove that the smooth-min B(x) satisfies B(x) ≤ min_i(b_i(x)).

Proof:

Let m = min_i(b_i). Then b_i ≥ m for all i, so $-\beta \cdot b_i \le -\beta \cdot m$, so $\exp(-\beta \cdot b_i) \le \exp(-\beta \cdot m)$.

Summing: $\Sigma_i \exp(-\beta \cdot b_i) \le N \cdot \exp(-\beta \cdot m)$.

Taking log: $\log(\Sigma) \le \log(N) + (-\beta \cdot m)$

Dividing by $-\beta$: $-(1/\beta) \cdot \log(\Sigma) \ge m - \log(N)/\beta = \min_i(b_i) - \log(N)/\beta$

Flip: $B(x) = -(1/\beta) \cdot \log(\Sigma)$ ... wait, this gives $B \ge \min - \log(N)/\beta$ (lower bound), not $B \le \min$.

For the UPPER bound: $\exp(-\beta \cdot m) \le \Sigma \exp(-\beta \cdot b_i)$ (just one term of the sum).

So $\log(\exp(-\beta \cdot m)) \le \log(\Sigma) \rightarrow -\beta \cdot m \le \log(\Sigma) \rightarrow -(1/\beta) \cdot \log(\Sigma) \le m$

Therefore: $B(x) \le \min_i(b_i(x))$. ✓

Combined: $\min - \log(N)/\beta \le B \le \min$.

For N=5, β=1000: $\log(5)/1000 \approx 0.0016$m = 1.6mm. B is within 1.6mm below the true min.

Q11: Why does $\dot{V} \le -\alpha \cdot V$ imply exponential convergence?

Proof (Grönwall's inequality):

$dV/dt \le -\alpha \cdot V$

Divide both sides by V (positive): $(1/V) \cdot dV/dt \le -\alpha$

This is $d/dt[\log V] \le -\alpha$

Integrate from 0 to t: $\log V(t) - \log V(0) \le -\alpha \cdot t$

Therefore: $V(t) \le V(0) \cdot \exp(-\alpha \cdot t)$

With α=4: V halves every $t=\ln(2)/4 \approx 0.17$s. V reduces by factor 1000 in $t=\ln(1000)/4 \approx 1.7$s.

So tracking error $\|e\| \le \sqrt{(V(0)) \cdot \exp(-\alpha/2 \cdot t)} = \|e(0)\| \cdot \exp(-2t)$.

Starting 20mm off the path: $e(0)=0.020$m → $e(1.7s) < 0.020/\sqrt{1000} \approx 0.63$mm ✓

Q12: The CBF constraint is $\nabla B \cdot (f+u) \ge -\gamma \cdot B$. What class of function is $\gamma \cdot B$?

A: The right-hand side is $-\gamma \cdot B(x)$, which is a class-K function of B: linear in B with coefficient γ. This is an "exponential CBF" (also called class-K CBF or ECBF). The condition ensures: If B(x(t)) = 0 (on boundary), then $\dot{B} \ge 0$ (can't decrease further → stays on or above boundary). If B(x(t)) > 0 (interior), then $\dot{B} \ge -\gamma B$ (can decrease, but not faster than exponential). This guarantees $B(x(t)) \ge B(x(0)) \cdot \exp(-\gamma \cdot t) \ge 0$ if $B(x(0)) \ge 0$. With γ=3: B decays at most as $\exp(-3t)$ — halves every 0.23s. But since the QP finds u that satisfies the constraint, B stays ≥ margin in practice.

Category 4: Application / Practical Questions

Q13: How do you handle an obstacle that the robot has NEVER seen before (new shape not in the 5 training types)?

A: This is a genuine limitation. The network is trained on 5 shape types (star, crescent, blob, kidney, lshape) with augmentation over pose/scale. A completely new shape type (e.g., a triangle) would make the encoder produce an out-of-distribution embedding → the CBF may be inaccurate. The analytic safety filter (which uses the KNOWN scene SDF) would still catch penetrations, but the learned flow field might not navigate around the new shape gracefully. Fix: add training examples for the new shape type and re-run Stage 2 fast retraining (3-5 minutes).

Q14: If an obstacle moves quickly (e.g., 0.5 m/s), does the controller still work?

A: The predictive look-ahead (LOOKAHEAD=0.10s in dynamic_env_test.py) helps: we install the obstacle at its predicted future position (0.1s ahead) when computing the CBF. For slow motions (<0.05 m/s as in the 4 test motions), this provides sufficient lead time. For fast obstacles (0.5 m/s), the needle would need to respond faster than its CBF allows — if the obstacle can close the safety margin in less than $1/(\gamma \times B_{initial}) \approx 1/(3 \times 0.04) \approx 8$ seconds, the CBF might fail. For high-speed dynamic obstacles, you need a higher γ or a HOBF (Higher-Order CBF), which is future work.

Q15: The eikonal term caps $\|\nabla B\| \leq 12$. But the ideal SDF gradient has $\|\nabla SDF\| = 1$, not 12. Why K=4 and cap at 12?

A: We scale the SDF by K=4 in the regression target: b_target = K·clip(sdf−Δ, ...) = 4·sdf. By the chain rule: $\nabla b_{target} = K \cdot \nabla sdf \rightarrow \|\nabla b_{target}\| = 4 \times 1 = 4$. So the ideal trained B should have $\|\nabla B\| \approx 4$ in the exterior (not 1). The eikonal cap at 12 = 3K is a soft ceiling: it penalizes gradients above 12, allowing values up to 12 without penalty. This gives 3× the ideal gradient as headroom for the boundary layer where $\|\nabla B\|$ might temporarily spike above 4. Setting the cap AT 4 would be too tight — even small training noise would cause penalties. Setting it at 12 catches true pathological cases (the near-step $\|\nabla B\|=290$ bug) without interfering with healthy training.

Q16: Explain in simple terms why "head-on deadlock" happens and how SWIRL_GAIN=1 fixes it.

A: Imagine a wall directly in front of you. ∇B points straight back (away from wall), directly OPPOSING the demo flow (which wants to go forward through the wall). In the QP: the CBF says "push backward along ∇B", the CLF says "push forward toward demo path." These are exactly opposite. With SWIRL_GAIN=0, the QP finds u that EXACTLY cancels f in the obstacle direction → u pulls needle left but demo flow pushes right → nearly zero net velocity → needle stalls dead in front of wall.

With SWIRL_GAIN=1: add a tangential component g (perpendicular to ∇B, pointing toward goal). Now the "forward" component is replaced by a "sideways" component. The needle starts circling around the obstacle. The CBF still says "don't go closer to the wall" — but "sideways" doesn't go closer, so no conflict. The CLF

slack ε absorbs the temporary increase in tracking error as the needle goes around. Eventually the needle clears the obstacle and the demo flow takes over again.

Q17: What is project_safe doing that the continuous CBF QP doesn't already handle?

A: The QP enforces the CONTINUOUS-TIME condition $\dot{B} \geq -\gamma B$. But we integrate with a DISCRETE step dt=0.01s. Even if $\dot{B}$ is correctly $\geq -\gamma B$ at the current moment, the actual change ΔB over dt might be more negative than expected because: (1) B is nonlinear — the gradient at x doesn't perfectly predict B at x+dt·v (2) The learned B has approximation errors project_safe directly checks: will $B(x + dt \cdot v) \geq$ margin? If yes, fine. If no, scale down v (bisection) until yes. It's a "trust but verify" step: trust the QP result but verify the actual next-step barrier value. This is the piece that gives strict discrete-time safety guarantees even with an imperfect learned barrier.

Q18: Can you retrain the system for a completely different task (e.g., different start/goal)?

A: Yes, with moderate effort. Stage 1 (~2.5h) trains f_θ on the new demo trajectories and V_θ for the new path — these are task-specific. Stage 2 (~5min) for B_φ is largely obstacle-specific and could potentially be reused if the obstacles are the same. In practice: if only start/goal changes but same obstacles, just retrain f_θ (Stage 1, Phase 0 only, ~30min). If new obstacle shapes appear, retrain B_φ (Stage 2, ~5min per new shape type). The modular f/V/B separation makes this feasible.

Category 5: Common Interview/Exam Questions on This Topic

Q19: Someone asks "What is the difference between a CLF and a CBF?" — explain in 3 sentences.

A: A CLF (Control Lyapunov Function) V(x) guarantees that the system converges to a target: $V>0$ everywhere except the goal, and the controller makes $\dot{V}<0$ (decreasing energy). A CBF (Control Barrier Function) B(x) guarantees that the system stays safe: $B \geq 0$ in the safe set, and the controller maintains $\dot{B} \geq -\gamma B$ (can't decrease too fast). In our system, CLF forces convergence to the demo path while CBF prevents penetrating obstacles — they are separate certificates enforced simultaneously via a single QP, with CLF being "soft" (can be violated) and CBF being "hard" (must hold).

Q20: Why is it important that the barrier B is "SDF-shaped" (not just positive/negative)?

A: The QP constraint uses ∇B — the gradient of B. If B is just a binary classifier (+1 outside, −1 inside, step function), then $\nabla B \approx 0$ everywhere except a tiny region at the boundary. In the QP: gradB·u ≥ cbf_rhs → with gradB ≈ 0 everywhere, the constraint becomes $0 \geq$ cbf_rhs, which is either always trivially satisfied (if cbf_rhs ≤ 0) or infeasible (if cbf_rhs > 0). Either way, the QP provides no useful correction. An SDF-shaped B has $\|\nabla B\| \approx K=4$ throughout, providing a meaningful direction to push in at any distance from the obstacle — this is what makes the QP work.

Q21: What would happen if LAMBDA_B_OB (unsafe hinge weight) were too small?

> A: If λ_OB is too small, the loss doesn't strongly penalize the network for predicting positive (safe) values inside obstacles. The network might satisfy the (cheaper) L_safe regression term by learning a large positive B everywhere, accepting small interior penalties. Result: B > 0 inside obstacles → the QP thinks we're safe when we're not → no avoidance correction → needle penetrates. λ_OB = 12 is high because safety violations are critical — we tolerate imprecise positive values (L_safe weight 5) but not missed unsafe detections (L_OB weight 12).

Q22: If you had to explain this project in 1 minute to a professor who doesn't work in this area:

> "We're building a robot controller that navigates a needle through foam to reach a target, while strictly avoiding critical tissue regions of irregular shapes. The key challenge is that these obstacles can appear at any angle or size — we can't pre-program every configuration. Our solution: train three neural networks — one to follow demonstration trajectories, one to measure how far we are from the path, and one to measure how close we are to obstacles — using random pose/scale transformations during training so the networks learn shape understanding rather than memorizing specific poses. At runtime, we solve a small quadratic optimization 100 times per second to compute the safest velocity that stays close to the demo path while respecting the obstacle boundaries. We've verified this achieves zero penetrations across all tested configurations including never-before-seen obstacle poses."

— End of Document —

Generated from source at /home/stanny/franka_ros2_ws/src/Tolerance_Aware_CBF(TA-CBF)/